



Hinweis

Durch die Größe der ROS2-Packages und Bagfiles wurden diese nicht angehängt. Die Code-Lösungen für die IMU-Auswertung und das Scanmatching (BasicICP.cpp) liegen als beigefügte Dateien vor.

Aufgabe A3.1 (Bagfiles)

1. Bagfile abspielen und visualisieren

Um die aufgezeichneten Odometrie-Daten erneut wiederzugeben und in RViz zu betrachten, wird das Bagfile in einem Terminal gestartet:

```
ros2 bag play <name_des_bagfiles> --loop
```

2. RViz2 konfigurieren

- **Fixed Frame:** Um eine Pfadhistorie im globalen Raum darstellen zu können, wird der Fixed Frame in RViz oben links auf `odom` gesetzt.
- **Odometry Visualizer:**
 1. Klick auf **Add -> By topic -> /odom** (Odometry) auswählen.
 2. In den Einstellungen des Visualizers kann die Darstellung der Pfeile (Toleranz, Farbe, Länge) konfiguriert werden, um den gefahrenen Weg als Vektoren sichtbar zu machen.
- **Ergebnis:** Siehe beigefügtes Bild A3.1.png .

Aufgabe A3.2 (IMU Positionsschätzung)

Berechnung der Position (Integration)

Um aus den Rohdaten der IMU (/imu/data_raw) eine X/Y-Position zu erhalten, müssen die Beschleunigungswerte und Gierraten iterativ über die Zeitdifferenz Δt aufintegriert werden:

1. **Geschwindigkeiten berechnen:**
 - $v_x = v_{x,alt} + a_x \cdot \Delta t$
 - $v_y = v_{y,alt} + a_y \cdot \Delta t$
2. **Betrag des resultierenden Geschwindigkeitsvektors:**
 - $v = \sqrt{v_x^2 + v_y^2 + v_z^2}$
3. **Drehwinkel berechnen:**
 - $\theta = \theta_{alt} + \omega_z \cdot \Delta t$
4. **Neue Pose berechnen:**
 - $x = x_{alt} + v \cdot \cos(\theta) \cdot \Delta t$
 - $y = y_{alt} + v \cdot \sin(\theta) \cdot \Delta t$

Vergleich: Odometrie vs. IMU-Trajektorie

Manöver: Es wurde versucht, eine einfache Kreisbahn innerhalb des Raumes zu fahren.

1. IMU-Trajektorie (imu_plot.png):

- Erwartungsgemäß ist die IMU zur alleinigen Positionsbestimmung völlig unbrauchbar.

2. Odometrie-Trajektorie (odom_plot.png):



- **Erkenntnis:** Die Odometrie bleibt im Gegensatz zur IMU in einem realistischen Maßstab.
- **Kritische Analyse (Fehlerfall):** Die geplottete Trajektorie spiegelt allerdings keine saubere Kreisbahn wider. Der Pfad ist extrem zackig, überlappt sich chaotisch und zeigt unphysikalische Sprünge.
- **Mögliche Ursachen für das Scheitern:**
 - Sensorik: Starke Rauscheffekte oder Aussetzer in den Odometrie-Daten, z.B. durch schlechte Rad-Encoder.
 - Aufzeichnung: Eventuell wurden beim Bag-Recording Nachrichten übersprungen (`dropped messages`) oder die Zeitstempel waren asynchron, wodurch Gnuplot zeitlich weit entfernte Punkte mit geraden Linien verbunden hat.

Aufgabe B3.1 (Scanmatching mit ICP)

Ziel: Ermittlung der räumlichen Transformation (t_x, t_y, θ) zwischen einem aktuellen Laserscan (`/scan`) und einem Referenzmodell (`/model`), sodass diese optimal deckungsgleich sind.

1. Initiale Transformation

Zur Visualisierung der initialen Verschiebung beider Scans wird ein statischer Transform-Publisher gestartet:

```
ros2 run tf2_ros static_transform_publisher 0 0 0 0 0 0 model laser
```

Hinweis: Dieser Befehl wird deaktiviert, sobald unsere eigene ICP-Implementierung Transformationen publisht, um TF-Konflikte zu vermeiden.

2. Closest Point Heuristik (Korrespondenzen)

- Für jeden Punkt im aktuellen Datenscan wird der euklidisch nächste Nachbar im Modellscan gesucht (Funktion `calcICPCorrespondences`).
- Zur Visualisierung in RViz werden diese Punktpaare mit einer `visualization_msgs/Marker` Message vom Typ `LINE_LIST` als rote Verbindungslinien gezeichnet.

3. Iterative Closest Point (ICP) nach Lu/Milios

Basierend auf den gefundenen Korrespondenzen wird die optimale Transformation in geschlossener Form berechnet (Funktion `calcTransformation`):

1. **Schwerpunkte berechnen:** c für Modellpunkte, c' für Datenpunkte.
2. **Kreuzkovarianzen bilden:** $S_{xx}, S_{yy}, S_{xy}, S_{yx}$.
3. **Rotation bestimmen:** $\theta = \arctan\left(\frac{S_{yx} - S_{xy}}{S_{xx} + S_{yy}}\right)$
4. **Translation bestimmen:**
 - $t_x = c_x - (c'_x \cos(\theta) - c'_y \sin(\theta))$
 - $t_y = c_y - (c'_x \sin(\theta) + c'_y \cos(\theta))$

4. Iteration & TF-Publishing

- Die gefundene Transformation wird auf den Datenscan angewendet (`transformPointCloud`).
- Der Prozess (Nachbarn finden → Lu/Milios anwenden → Transformieren) läuft über einen asynchronen Timer mit einer Rate von 0.25 Hz (alle 4 Sekunden).



- Nach jeder Iteration wird der aufsummierte Gesamtfehler `eps` im Terminal ausgegeben.
- Die kumulierte Transformation wird über den `tf2_ros::TransformBroadcaster` als `tf`-Frame (von `model` nach `laser`) in das ROS-System gepublikt. Der Fortschritt der Deckungsgleichheit ist live in RViz sichtbar.
- **Ergebnis:** Siehe beigefügtes Bild `B3.1.png`.
- **Code-Lösung:** Siehe beigefügte `BasicICP.cpp`.